

RELATÓRIO DE PROJETO

Projeto: Missão Marte - Unifor

Disciplina: Proj arquitetura de sistemas

1. INTEGRANTES DA EQUIPE

- **Nome do Aluno 1:** João Pedro Nobre Sousa
- **Nome do Aluno 2:** Paulo Levi Pontes Furtado

2. RESUMO DO PROJETO

O projeto consiste no desenvolvimento de um jogo espacial em ambiente de terminal Java. O jogador controla uma nave em um mapa bidimensional com o objetivo de resgatar passageiros e retornar com segurança à plataforma de pouso na coordenada central (0, 0).

A aplicação consolida conceitos essenciais de Orientação a Objetos (Encapsulamento, Herança, Polimorfismo e Enums), algoritmos de colisão, movimentação de entidades e manipulação de arquivos externos em formato JSON para persistência de dados.

3. ATIVIDADES E EXERCÍCIOS REALIZADOS

- **Exercício 1 (Modificação de Atributos):** Ajuste na capacidade máxima de carga da nave na classe `Nave` para permitir até 5 passageiros simultâneos.
- **Exercício 2 (Hierarquia de Classes):** Criação da classe `Astronauta` herdando da classe base `Passageiro`, implementando o reuso de código e especialização de entidades.
- **Exercício 3 (Aparência do Mapa):** Alteração na renderização visual do mapa na classe `Missao`, atualizando os símbolos para @ (Nave) e # (Asteroides).
- **Exercício 4 (Polimorfismo e Pontuação Bônus):** Implementação de polimorfismo através do método `getPontuacao()` sobrescrito (`@Override`) em cada tipo de passageiro:
 - **Professor:** +10 pontos
 - **Engenheiro:** +15 pontos
 - **Astronauta:** +20 pontos
- **Exercício 5 (Sistema de Vidas):** Adição de controle de danos na nave. Ao colidir com obstáculos, o jogador perde 1 vida e é redefinido para a origem (0, 0). O jogo encerra em *Game Over* caso as vidas cheguem a 0.
- **Exercício 6 (Mapa Expandível):** Implementação de dimensão dinâmica para o mapa espacial via entrada do usuário (`Scanner`), permitindo definir coordenadas de -X a +X.

- **Exercício 7 (Inimigos com IA):** Criação da classe `Inimigo` com algoritmo de movimentação aleatória a cada turno e verificação de colisão durante a exploração.
- **Exercício 8 (Menu de Dificuldade):** Implementação da `enum Dificuldade` (FÁCIL, MÉDIO, DIFÍCIL), ajustando dinamicamente a quantidade de asteroides, pontos/combustível iniciais e capacidade.
- **Exercício 9 (Persistência em JSON):** Criação da classe `RankingEntry` e manipulação de arquivos para gravar e ler o histórico de pontuações, incluindo data/hora, passageiros coletados e nível jogado no arquivo `ranking.json`.
- **Exercício 10 (Fluxo de Pouso Lunar e Menu Completo):**
 - **Mecânica Lunar:** Adição do símbolo `L` em `(0, 0)`. Após resgatar a tripulação, o jogador deve retornar à base, consumindo 1 ponto de combustível por movimento.
 - **Estatísticas:** Medição do tempo de partida em segundos (`System.currentTimeMillis()`) e contagem total de movimentos.
 - **Menu Principal:** Implementação de navegação interativa com opções de Novo Jogo, Top 5 Ranking, Reset de Histórico e Sair.

4. CONCLUSÃO

Todas as funcionalidades solicitadas do Exercício 1 ao 10 foram implementadas com sucesso. O sistema apresenta código modularizado, tratamento de colisões e regras de encerramento coesas, cumprindo todos os requisitos estabelecidos na especificação.